

std::cstring_view

Document #	P3655R3
Date	2025-10-05
Targeted subgroups	LEWG, LWG
Ship vehicle	C++29
Reply-to	Peter Bindels <dascandy@gmail.com> Hana Dusíková <hanicka@hanicka.net> Jeremy Rifkin <jeremy@rifkin.dev> Marco Foco <marco.foco@gmail.com> Alexey Shevlyakov <aleshevl@gmail.com>

§ 1 Abstract

We propose a standard string view type that guarantees null-termination.

§ 2 Introduction

C++17 introduced `std::string_view`, a non-owning view of a continuous sequence of characters. It is cheap to use, offers fast operations, and replaced most uses of `const std::string&` or `const char*` as function parameters. The utility and interface of string views as well as the benefits of not having to do unnecessary `strlen` calculations on C-style strings makes string view types highly desirable for working with strings in C++.

Unlike `const std::string&` and many but not all `const char*`, `std::string_view` is not null-terminated. This allows it to have fast and cheap substring operations. However, this also means `std::string_view` is not a suitable replacement for either of the two aforementioned types whenever a null-terminated C-style string is needed. While most C++ code mostly interfaces with C++ code, it is not uncommon to need to use operating system calls, C interfaces, third-party library APIs, or even C++ standard library APIs which require null-terminated strings. Because of a lack of a desirable option for passing non-owned null-terminated strings, `std::string_view` parameters are none the less sometimes used today in cases where null-terminated strings are needed, calling `std::string_view::data` to get a `const char*`. This is, needless to say, very bug-prone.

For this reason, many C++ developers use custom `cstring_view` or `cstring_view` types which are guaranteed to be null-terminated. Its wide presence on Github with implementations from among others Microsoft, Google, and many smaller projects, indicates a wide support for the type. As it's a lingua franca type, it should be part of the standard C++ library.

§ 3 Revision History

§ 3.1 R0, February 2025

§ 3.2 R1, May 2025

- Update with new numbers about use on github.
- Integrate SG16 feedback
- Add approaches to prioritize or select a particular overload for multiple competing overloads of a function taking a string-like argument
- Number constructor overloads for discussion ease
- Add polls on `char_traits` and contained NULs

§ 3.3 R2, June 2025

- Merge with P3710 from Marco Foco
- Expand on constructor rationales

§ 3.4 R3, October 2025

- Add revision history
- Reorder chapters into "discussion on type", "design rationale", "historical information" and "wording" for easier referencing and more structured reading
- Remove polls section that was created for Sofia, as we now have answers.
- Narrow paper based on feedback from Sofia SG23 and LEWG
- Add reference implementation on Beman Project

- Expanded explanation on constructors
- Removed nullptr constructor (novel, should be separate paper), added empty constructor (accidental omission)
- Reword relation between `string_view` and `cstring_view` after information from Corentin Jabot about constructors being preferred over conversion operators
- Expanded wording

§ 4 Previous Papers

The idea of `cstring_view` was present even in the original paper for `string_view` (Sept 2012 - Feb 2014) <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3921.html#null-termination>:

Another option would be to define a separate `cstring_view` class to represent null-terminated strings and let it decay to `string_view` when necessary. That's plausible but not part of this proposal.

An attempt was made in Feb 2019 to concretely propose the type (renamed to `cstring_view`) with <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1402r0.pdf>, but it failed to gain consensus in LEWG at <https://wiki.edg.com/bin/view/Wg21kona2019/P1402>. In fact, the discussion there concluded with "CONSENSUS: We will not pursue P1402R0 or this problem space." It is also worth noting that contracts were briefly contemplated in the minutes as a way of addressing the problem, however, contracts can't check for a null character safely as it wouldn't be part of the view. Similarly, other runtime checks for `string_view` null-termination are off the table.

However, in 2024 <https://wg21.link/p3081> was published which contains an aside regarding a null-terminated `cstring_view` in section 8, noting:

This is one of the commonly-requested features from the <https://github.com/microsoft/GSL> library that does not yet have a `std::` equivalent. It was specifically requested by several reviewers of this work.

During the discussion, it was made clear that `cstring_view` deserves attention on its own and it should not be happenstance introduced as part of an entirely different topic. While P3081 did not end up passing for reasons other than `cstring_view`, it is another example of the utility being seen as desirable.

We also have <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2996r9.html>, which in its specification of functions returning a `string_view` or `u8string_view` tries its best to say it's actually supposed to be a `cstring_view`, except without naming the type for existential reasons:

Any function in namespace `std::meta` that whose return type is `string_view` or `u8string_view` returns an object `V` such that `V.data()[V.size()] == '\0'`.

As such, we do believe that there is both space for such a type, and a desire from multiple angles to have it defined in the standard library.

§ 5 Discussion on the type

§ 5.1 Reasons to have the type

Many functions right now whether C++ standard library calls, operating system calls, or third-party library calls require null-terminated C-style strings. Absent an efficient type that can represent a non-owning view of a null-terminated string, these functions must take either a `const char*` and incur potential `strlen` overhead, take a `const std::string&` and possibly superfluously copy and allocate, take a `string_view` and make a copy, or haphazardly take a `std::string_view` with a fragile unenforceable contract that it is a view of a null-terminated string. Such a contract would be unenforceable because `pre(sv[sv.size()] == 0)` is potentially undefined behavior.

String views tend to start their life coming from a string literal or from a type that owns its internal buffer, often `std::string`. Both of these are places that can always create a `cstring_view` instead, as they both know their data is inherently null terminated.

Strings are very often just passed along with a non-owning string type all the way to where they are used as data. Some of these potential uses, such as calling `std::ofstream::write`, do not rely on the null terminator, but others, like `std::ofstream::ofstream`, do. Using `string_view` as the intermediate type loses the knowledge that a null terminator is already guaranteed to be present, requiring the developer to either make assumptions or make a copy. Another common use might be creating a stringstream from a string <https://stackoverflow.com/questions/58524805/is-there-a-way-to-create-a-stringstream-from-a-string-view-without-copying-data>.

We do not have to look far for examples of `std::string_view` being used in bug-prone ways with APIs expecting null-terminators. Searching `/\.data\(\)/ string_view language:c++ -is:fork` on GitHub code search turns up two examples on the first page:

From <https://github.com/surge-synthesizer/shortcircuit/blob/ba6ea3a2e703e4fa0ed069427aa42b668699b624/libs/md5sum/demo.cc#L37>

```
std::optional<std::string> hash_file(const std::string_view &file_name) {
    auto fd = open(file_name.data(), O_RDONLY);
    ...
}
```

From https://github.com/VisualGMO/gmq_header/blob/cbc2853f391acc51ddd25a50d567cac404776413/log.hpp#L66

```
void log(Level level, std::string_view funcName, std::string_view filename, unsigned int line, Args&&... args) {
    if (level <= level_) {
        printf("[%s][%s][%s][%u]", Level2Str(level).data(), filename.data(), funcName.data(), line);
        ...
    }
}
```

These two examples could be argued to be bad code or misuses of `std::string_view`, however, the fact that they are written reflects the desire for the ability to use string view types in such cases.

These problems are visible enough that we have targeted patches for specific instances of this problem in the standard - <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2495r0.pdf> attempts to directly patch this specific example. The solution bypasses that the problem is that we're missing an unbroken type-safe chain of knowledge that the value being passed is or is not null-terminated.

The type effectively fills out the design space that exists around strings within C++. We started off with `std::string` as an owning string type, ambiguously being specified as having a null terminator or not, clarified in C++11 to definitely *have* a null terminator. C++14 added `std::string_view` to that set, offering a non-null-terminated non-owning string type. The type itself raises the question, should we add a non-null-terminated owning string type, and/or a null-terminated non-owning string type? We're proposing the last of these, leaving only the non-null-terminated owning string type as not present. We believe there is no situation to be written where a non-null-terminated owning string type would have a measurable benefit over the null-terminated owning string type that we have, and as such it is not worth the added complexity.

Without them we retain the question we had before - `const char*`, `const std::string&` or `string_view`? The first loses lots of type safety and potentially requires redundant `strlen` computation elsewhere in the software, the second requires it to be a `std::string` but retains null-termination knowledge, the last offers the ability to send any string type constructible to a `string_view`, but loses any knowledge of null termination. For the common case of passing along "some string input" to a function that ends up requiring null termination, the proposed `cstring_view` is the only type that properly captures it.

Since the first draft of this paper the use of `cstring_view` and `string_view` on Github has grown from 1.9k to 2.1k, indicating active use, and in many cases people adding new implementations of the type. Many contain the subtle bugs that we highlight in this paper, illustrating why it is a good idea to add the type to the standard.

§ 5.2 Reasons not to have the type

In an ideal world, we could actually fix the operating systems and third-party libraries to accept pointer-and-size strings in all places, removing the need for null termination to exist, and for null termination propagation to be relevant. Sometimes, in particular from environments where this may not be unrealistic in a subset, the argument is voiced that `cstring_view` does not offer any benefits.

Adding the type complicates the type system around strings by having more options for string types. This is not as much of an argument as it seems, since for each site a single type is the most appropriate, and the only places that would differ are those where the difference can make a meaningful performance impact - ie, the exact kind of tight loop where the type is useful for being able to make the difference.

§ 6 Design rationale

§ 6.1 `cstring_view` and `string_view` relation

`cstring_view` should be transparently convertible to a `string_view`. There are multiple ways to create this conversion:

1. Implementing `cstring_view` as a subclass of `string_view`
2. Implementing `string_view` as subclass of `cstring_view`
3. Adding a conversion operator from `cstring_view` to `string_view`
4. Adding a constructor to `string_view` from `cstring_view`

For 1 and 2, the main question is if either of these types has a set of properties that is a strict subset of the other (ie, LSP). `cstring_view` has a guarantee that `string_view` does not uphold, so `string_view` cannot inherit publicly from `cstring_view`. However, `string_view` offers an operator= from `string_view`, which `cstring_view` cannot support, so `cstring_view` cannot inherit publicly from `string_view` either. That leaves the latter two strategies, and preference is given to the constructor approach.

This brings into reality an overload ambiguity:

```
void handle(string_view v);
void handle(cstring_view v);
```

```
handle("Hello World!");
```

This is now an ambiguous function call. It is not a new problem; <https://godbolt.org/z/c31e31fKW> shows that the exact same problem already happens with regular `std::string` and `std::string_view`. The reason is somewhat fundamental; all three types model the concept "string" and `*are*` ambiguous. The user should be clear about which properties of string it's expecting. Adding this new type only adds to the vocabulary the option to say "non-owning null-terminated", giving the user better choices rather than complicating it.

§ 6.1.1 Deprioritizing one overload

When one overload is preferred, but others should exist, the less preferred ones can be marked with `requires true`. This lifts one overload up out of the overload set for ambiguous matches, while leaving the rest to exist.

§ 6.1.2 Tag type forced conversions for exact matches

<https://godbolt.org/z/8qssnq8rf> and <https://godbolt.org/z/xG7955hf9> illustrate the idea of using a wrapping template to make one overload stand out.

§ 6.1.3 Explicitly adding a function to disambiguate manually

<https://godbolt.org/z/9zoGEh1cv> demonstrates adding an overload that matches anything that isn't an exact match, which indirections to a function that hand-picks an overload.

§ 6.2 Construction

`std::basic_string_view` offers a handful of constructors. The principle behind which to support is that if we can match the interface of `string`, we should, and if not, if we can support the behavior of `string_view` we should.

```
constexpr basic_cstring_view(); // (0)
```

Creates a `cstring_view` that refers to a static null terminator. `data()` and `c_str()` are valid to call and result in a zero-length string, `size()` returns zero. Behavior matches `string`.

```
constexpr basic_cstring_view(const charT* str); // (1)
```

The basic constructor from an unadorned `charT*` risks taking a non-null-terminated string and searching for the null terminator. For `cstring_view` this constructor is as safe as it is for `string_view`, since we need to have a null terminator anyway.

```
constexpr basic_cstring_view(const charT* str, size_type len); // (2)
```

This constructor conceptually offers a $O(1)$ construction time. For `cstring_view` we add a contract that asserts that `str[len] == '\0'`. The behavior is logically identical to `string` except that we require `str` to be dereferenceable and to dereference to a nul terminator.

```
template<class It, class End>
constexpr basic_cstring_view(It begin, End end); // (3)
```

This constructor looks more difficult to support, but `string_view`'s requirements on this constructor make it possible to support in the same fashion. From the action of creating a `cstring_view` we get the implication from the user that the iterators are contiguous and point to a sized range, and that it must be nul-terminated. We specify the contract that `*(begin + (end - begin)) == '\0'`, and that the contiguous range specified from `begin` must be dereferenceable in `[begin ... (begin + (end - begin))]`.

```
template<class R>
constexpr explicit basic_cstring_view(R&& r); // (4)
```

This constructor has the same implication as (3) in that the range passed in must end with a nul terminator. The ability to check `data()[size()]` is rare, but is specified on `string`. If we have any other range though, they are always specified in being dereferenceable only from `0` to `size()-1`, but not at `size()`. We keep this constructor with a requirement on the type `R` that it satisfies the concept of having a nul terminator at `size()`, and being dereferenceable. - or do we want a requirement that it has a `c_str` function? Either way, restrict it on the concept of what it needs to work.

§ 6.3 Member functions on `string_view` that return a `string_view`

In some cases the functions can be replicated on `cstring_view` with the return type being a `cstring_view`, while in some cases the return type loses the ability to guarantee null termination, and should still return a `string_view`. The types are entangled. Note that the user can always treat the return values as-if they were `string_view` as the type is implicitly convertible; this is strictly giving the user more expressivity rather than less.

§ 6.3.1 `substr`

The `substr` function is changed from a single function with a default argument, to two functions, one with one and one with two arguments. The one-argument `substr` always retains the end (identical to `string_view`'s `substr`) and returns a `cstring_view`; the two-argument `substr` at least in a conceptual sense chops off at least the null terminator from the end, and should always return a `string_view`. If the user cares about this difference they can assign to a `cstring_view` and get a compile error if their call is incompatible. If they assign to a `string_view` the behavior is identical to what `string_view` would have done.

§ 6.3.2 `subview`

The `subview` function is newly added in the 2025 Sofia meeting, which always creates a view on a part of the input string. In this paper we treat it the same as the `substr` function and create a `cstring_view` if we can maintain the null termination guarantee, or create a `string_view` if we cannot statically maintain the guarantee. There is an adjoint paper (P3862) that proposes to fix this in the existing-but-unpublished C++26 standard to avoid future deviation in having `std::string::subview` return `string_view` for `subview(n)`.

§ 6.3.3 `remove\_prefix`

`remove_prefix` drops characters from the front of the string it's called on. Behavior fits.

§ 6.3.4 `remove\_suffix`

`remove_suffix` changes the string in-situ, and as such is unimplementable on `cstring_view`. It should be a function marked as `=delete` with a reason pointing users to use the two-argument `substr` function instead, which returns a `string_view`.

§ 6.4 Embedded NUL bytes

The practice of having an embedded NUL character in a `string` or `string_view` is very uncommon, and many developers do not consider such strings when writing their code. The use case of having a null-terminated string is much more common, in particular in interaction with C APIs (such as OS APIs, things like the Yubico libfido2 library, libsqlite and others). We have the option of forbidding and/or checking for embedded NULs in `cstring_view`.

+ Prevents runtime shorter actual string length

- Creates unnecessary incompatibility with `string` and `string_view`
- Makes conversion from `string` to `string_view` different from conversion from `string` through `cstring_view` to `string_view`.
- Adds an $O(n)$ check to all $O(1)$ constructor
- Actual prevalence of problem being prevented is minimal

We do not propose to forbid embedded NUL bytes, keeping the type aligned with `string` and `string_view`. The existence of embedded NUL bytes is a separate topic and should be handled in a separate paper. Note that even the C++ standard itself has embedded NULs in some of its `string` values, for example in `[facet.numpunct.virtuals]/3 do_grouping`.

§ 6.5 Lazy size calculation?

The type is used to carry the type-wise annotation that an existing allocated string is null-terminated to a place that needs this information. `string` and `string_view` eagerly calculate the size and store it. Both of these types are typically used in C++-style usage and need the size to check bounds, as well as to provide the user with size information with the `size()` function.

For `cstring_view`, it can be argued that the main use of the type is to eventually still have the type-wise knowledge that `c_str()` exists and gives a null-terminated string, and that use as a C++-style string is secondary. In that same argument, eager calculation of `size()` is of no value, since the user expects only to call `c_str()` eventually, getting less or no benefit from the `size()` calculation other than the safety guarantee that on construction the value was null-terminated.

To enable this, we could change the (1) constructor to be $O(1)$ and to set the internal size to a static invalid value. The `size()` function would change to amortized $O(1)$ and on first call it would store the calculated size in the member value. All other functions that rely directly or indirectly on `size()` being $O(1)$ would change to being amortized $O(1)$.

Doing so, we would lose the ability to put (hardened) preconditions that have anything to do with the size on any function. The constructor wouldn't be able to check that it's indeed a valid range with a null terminator. `operator[]` can check that it's within range in a hardened precondition and has to be demoted to amortized $O(1)$, or it stays $O(1)$ and we cannot add that precondition as hardened.

Computing the `size()` at construction gives an additional safety guarantee: the eagerly-computed size can be used as a hard limit for string iteration, and enables checking that the buffer has not been compromised, thus limiting the risk for unbounded string iterations (that are a common sources of CVEs - Common Vulnerabilities and Exposures).

The change seems strongly related to whether the language we're writing is C or C++ first. If we're writing C, then we should be opting for the direction that does not calculate size eagerly, and potentially not at all. If we're writing C++, the solution in line with what C++26 has added for hardened preconditions is to accept the size calculation. Users that really cannot afford that are still able to use `const char*`.

§ 6.6 Conversion from string being explicit?

The rationale for this would be that a `string` can contain embedded NULs, and this would make explicit that we're switching to a type that really doesn't like those. The downside of this is that we are introducing needless friction into the most common usage path, where the vast majority of users never use strings with embedded NULs.

§ 6.7 Modifying the `cstring_view` or underlying storage

`string_view` itself does not allow modifying the values through its interface and `cstring_view` should not deviate from this. `string_view` only states that the underlying storage must outlive the `string_view` built on it, and `cstring_view` will do the same, with the addition that the null terminator that it requires may not be modified, otherwise we get library UB. Adding or removing a NUL inside of the string has the same behavior as it does in `string_view` - you get an embedded NUL. That is a thing we can work to prevent, in a separate paper.

§ 6.8 Standard Library Changes

There are a handful of interfaces in the standard library which take `const string&` and really want a `cstring_view` or possibly a `string_view`. These include:

- Constructors for `std::except` types, `system_error`, `format_error`, `ios_base::failure`, and `std::filesystem::filesystem_error`
- The `stoi` family of functions
- `random_device::random_device`
- Lots of locale interfaces
- `basic_filebuf::open`, `basic_ifstream::basic_ifstream`, `basic_ifstream::open`, `basic_ofstream::basic_ofstream`, `basic_ofstream::open`, `basic_fstream::basic_fstream`, and `basic_fstream::open`

There are further interfaces which take `const char*` and could have overloads taking a `cstring_view` or `string_view`.

We do not propose any such changes in this paper but would like to investigate this in a follow-up paper.

§ 6.9 Reference Implementation

A reference implementation is at https://github.com/bemanproject/cstring_view.

§ 7 Historically relevant information

§ 7.1 Bikeshedding

Null-terminated string view types are typically named `zstring_view` (N3921, P3081, GSL) or `cstring_view` (P1402). `cstring_view` follows the `std::string::c_str()` nomenclature while `zstring_view` has some establishment and recognizability.

Github code search shows similar popularity between https://github.com/search?q=%2F%5Cbcstring_view%5Cb%2F%20language%3Ac%2B%2B%20-is%3Afork&type=code (1.2k results as of the time of writing) and https://github.com/search?q=%2F%5Cbzstring_view%5Cb%2F+language%3Ac%2B%2B-is%3Afork&type=code (680 results as of the time of writing). In refreshing this paper, it appears that `cstring_view` has gained ~200 added results, while `zstring_view` only added 8, hinting at a popular preference for the former name. In the October 2025 update, `cstring_view` has gained another 300 results, with `zstring_view` gaining 208.

The paper proposes `cstring_view`, as it has minor number advantage in every measurement so far.

§ 7.2 Prior Polls

§ 7.2.1 April 2025, Online SG16

§ 7.2.1.1 Poll 1: P3655R0: No objection to use of `std::char_traits` for consistency and compatibility with `std::string_view`.

Attendees: 8 (no abstentions)

7/1/0/0/0

Strong consensus.

§ 7.2.1.2 Poll 2: P3655R0: Forward to LEWG with encouragement to add analysis of overload resolution and techniques to address ambiguity.

Attendees: 8 (no abstentions)

8/0/0/0/0

Strong consensus.

§ 7.2.2 June 2025, Sofia SG23

§ 7.2.2.1 11.4 We want a separate bounded array constructor (prioritized wrt char)*

6/5/0/0/1

Consensus

§ 7.2.2.2 11.5 We want to forbid constructing/assigning `zstring_view` containing embedded null chars

3/6/0/0/3

Weak consensus

§ 7.2.2.3 We want to drop the first constructor from `zstring_view` in this paper taking the char*)*

2/3/3/0/2

No consensus

§ 7.2.3 June 2025, Sofia LEWG

§ 7.2.3.1 POLL: We want to spend more time on `zstring_view`

23/8/5/0/0

Attendance: 30 (IP) + 11 (R)

Author's Position: 2xSF

Outcome: strong consensus in favour

§ 7.2.3.2 POLL: Rename `zstring_view` to `cstring_view`

8/6/13/4/2

Attendance: 30 (IP) + 11 (R)

Author's Position: F, A

Outcome: No consensus.

§ 7.2.3.3 POLL: `zstring_view` should be disallowed to have NUL characters in the middle.

2/6/10/4/11

Attendance: 27 (IP) + 11 (R)

Author's Position: F, A

Outcome: No consensus for change

§ 7.3 NVIDIA Experience (moved from p3710)

NVIDIA implemented `cstring_view` independently, with almost identical features. This is described in <https://wg21.link/p3710>, now merged into this paper. We also implemented some additional features described in <https://wg21.link/p3566>.

Ideally we wanted our input parameters to all be `string_view` and all the output parameter to be `cstring_view` (which gives more flexibility).

In practice, there are exceptions on both sides:

- When a null-terminated parameter is needed for internal reasons outside of our control, i.e. when interacting with system calls or third-party libraries, it's preferable to have a `cstring_view` parameter.
- When returning a part of an internal string, e.g. when extracting the file name from a full path, it's more convenient to just return a `string_view`.

Nonetheless, having a large codebase, we knew we couldn't just rewrite the entire codebase to apply this change, so we used `cstring_view` as a tool for steering our codebase in the ideal direction described above.

Initially, we used `cstring_view` as a drop-in replacement for `const char*` parameters and return values of our APIs, without worrying too much whether or not the function expects a null terminator. As a sidenote, for this migration we asked our developers to be intentional in using `.data()` or `.c_str()`, and use `.data()` when a null-terminator is not expected, and only use `.c_str()` when the string is required to be terminated. If respected, this rule allows us to just try changing a parameter from `cstring_view` to `string_view` and verify whether or not a function is relying on some parameter to be null-terminated. This operation can be done incrementally, one function at a time.

Once this was done, we started from the "deepest" functions to analyze their usage, and replace their parameters `cstring_view` to `string_view`, or rewrite them to allow for `string_view` parameters. We proceeded upwards in the call chain, and did the same. Again, this can be done in multiple passes.

What we observed is that, in some case, this might not only make the code safer, but also enable new optimizations.

Here follows an example of a parameter upgrade how we implemented these changes, step-by-step, with an example of how this can lead to more performant and safer code at the end of the process.

```
void f(const char* x) {
    external_library::g1(x);
}

void f1() {
    std::string a1 = "test";
    f(a1.c_str());
}

void f2() {
    constexpr char a2[] = "test";
    f(a2);
}

void f3() {
    const char* a3 = "test";
    f(a3);
}

void f4() {
    string_view a4 = "abcde";
    string_view subtext = a4.substr(2, 2); // no null terminator at the end of `subtext`

    f(string{subtext}); // create a temporary to add the terminator
}
```

In this example, `external_library::g1` is a placeholder for some generic computation that happens on the string.

Initially, we update the API we will change `f` to:

```
void f(cstring_view x) {
    external_library::g1(x.c_str());
}
```

All the functions now have well-defined memory ranges (no unbounded string).

We can now then look for alternatives, and find out there's a different API for the `external_library::g1` we can use, e.g. `external_library::g2(char*, size_t)`, that doesn't require a null-terminated sequence.

```
void f(string_view x) {
    external_library::g2(x.data(), x.size());
}
```

As next step, we can check all the usages, and see if we can get rid of some extra code, for example:

```
void f4() {
    string_view a4 = "abcde";
    string_view subtext = a4.substr(2, 2); // no null terminator at the end of `subtext`

    f(subtext); // f now accepts a string_view, no need for temporary
}
```

This results in a better-optimized code at the end (less need for temporary strings).

NOTE: The sequence of changes in our codebase was different, because our codebase includes the changes proposed in <https://wg21.link/p3566>, so some of the intermediate steps relying on implicit `char*` -> `string_view` and `char*` -> `cstring_view` conversions require additional changes, and intermediate steps marking conversions with the proposed `unsafe_length` tag.

§ 8 Wording

Changes in `[format.formatter.spec]` and `[string.view.general]` are deltas, all subsequent are full additions.

§ 8.1 `[format.formatter.spec]`

Add to 2.2:

```
template<class traits>
    struct formatter<basic_cstring_view<charT, traits>, charT>;
```

Add to 4.1:

```
template<class traits>
    struct formatter<basic_cstring_view<char, traits>, wchar_t>;
```

§ 8.2 String View Classes `[string.view]`

§ 8.2.1 General `[string.view.general]`

Update as indicated:

The class template `basic_string_view` describes an object that can refer to a constant contiguous sequence of char-like (`[strings.general]`) objects with the first element of the sequence at position zero. **The class template `basic_cstring_view` describes an object that can refer to a constant contiguous sequence of char-like (`[strings.general]`) objects with the first element of the sequence at position zero, that is guaranteed to contain a null-terminator at the position `size()`.** In the rest of `[string.view]`, the type of the char-like objects held in a `basic_string_view` **or a `basic_cstring_view`** object is designated by `charT`.

For `basic_string_view`, `[data(), data() + size())` is a valid range. For `basic_cstring_view`, `[data(), data() + size())` is a valid range and `data() + size()` points at an object with value `charT()` (a "null terminator").

[Note 1 : The library provides implicit conversions from `const charT*` and `std::basic_string<charT, ...>` to **`std::basic_cstring_view<charT, ...>`, and implicit conversions from `const char*`, `std::basic_string<charT, ...>` and `std::basic_cstring_view<charT, ...>` to `std::basic_string_view<charT, ...>`** so that user code can accept just `std::basic_string_view<charT>` **or `std::basic_cstring_view<charT>`** as a non-templated parameter wherever a sequence of characters is expected. User-defined types can define their own implicit conversions to `std::basic_string_view<charT>` **or `std::basic_cstring_view<charT>`** in order to interoperate with these functions. — end note]

[Note 2: `basic_cstring_view<charT, ...>` is primarily useful when working with APIs which expect null terminated strings. — end note]

§ 8.2.1.1 Header `cstring_view` `[string.view.cstring_view]`

```
namespace std {
    template<typename T>
    concept cstring_like = requires(const T & t) { { t.c_str() } -> std::same_as<const T::value_type*> };

    // [string.view.cstring_view.template], class template basic_cstring_view
    template<class charT, class traits = char_traits<charT>>
    class basic_cstring_view; // partially freestanding

    template<class charT, class traits>
        constexpr bool ranges::enable_view<basic_cstring_view<charT, traits>> = true;
    template<class charT, class traits>
        constexpr bool ranges::enable_borrowed_range<basic_cstring_view<charT, traits>> = true;

    // [string.view.cstring_view.comparison], non-member comparison functions
    template<class charT, class traits>
        constexpr bool operator==(basic_cstring_view<charT, traits> x,
                                   type_identity_t<basic_cstring_view<charT, traits>> y) noexcept;
    template<class charT, class traits>
        constexpr /* see below */
    operator<=>(basic_cstring_view<charT, traits> x,
                 type_identity_t<basic_cstring_view<charT, traits>> y) noexcept;
```

```

// [string.view.cstring_view.io], inserters and extractors
template<class charT, class traits>
    basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os,
                    basic_cstring_view<charT, traits> str);           // hosted

// basic_cstring_view typedef-names
using cstring_view      = basic_cstring_view<char>;
using u8cstring_view    = basic_cstring_view<char8_t>;
using u16cstring_view   = basic_cstring_view<char16_t>;
using u32cstring_view   = basic_cstring_view<char32_t>;
using wcstring_view     = basic_cstring_view<wchar_t>;

// [string.view.cstring_view.hash], hash support
template<class T> struct hash;
template<> struct hash<cstring_view>;
template<> struct hash<u8cstring_view>;
template<> struct hash<u16cstring_view>;
template<> struct hash<u32cstring_view>;
template<> struct hash<wcstring_view>;

inline namespace literals {
    inline namespace cstring_view_literals {
        // [string.view.cstring_view.literals], suffix for basic_cstring_view literals
        constexpr cstring_view      operator""csv(const char* str, size_t len) noexcept;
        constexpr u8cstring_view    operator""csv(const char8_t* str, size_t len) noexcept;
        constexpr u16cstring_view   operator""csv(const char16_t* str, size_t len) noexcept;
        constexpr u32cstring_view   operator""csv(const char32_t* str, size_t len) noexcept;
        constexpr wcstring_view     operator""csv(const wchar_t* str, size_t len) noexcept;
    }
}

```

§ 8.2.2 Class template basic_cstring_view [string.view.cstring_view.template]

§ 8.2.2.1 General [string.view.cstring_view.template.general]

Add this whole section

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_cstring_view {
    public:
        // types
        using traits_type      = traits;
        using value_type       = charT;
        using pointer          = value_type*;
        using const_pointer    = const value_type*;
        using reference        = value_type&;
        using const_reference  = const value_type&;
        using const_iterator   = implementation-defined; // see [string.view.cstring_view.iterators]
        using iterator         = const_iterator;
        using const_reverse_iterator = reverse_iterator<const_iterator>;
        using reverse_iterator = const_reverse_iterator;
        using size_type        = size_t;
        using difference_type  = ptrdiff_t;
        static constexpr size_type npos = size_type(-1);

        // [string.view.cstring_view.cons], construction and assignment
        constexpr basic_cstring_view() noexcept;
        basic_cstring_view(const basic_cstring_view&) noexcept = default;
        basic_cstring_view& operator=(const basic_cstring_view&) noexcept = default;
        constexpr basic_cstring_view(const charT* str) noexcept
            pre(str != nullptr);
        constexpr basic_cstring_view(const charT* str, size_type len) noexcept
            pre(str != nullptr)
            pre(str[len] == '\0');
    };
}

```

```

template<class It, class End>
constexpr basic_cstring_view(It begin, End end)
    pre (*(begin + (end - begin)) == charT());
template<class R>
constexpr basic_cstring_view(cstring_like R&& r);

// [string.view.cstring_view.iterators], iterator support
constexpr const_iterator begin() const noexcept;
constexpr const_iterator end() const noexcept;
constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr const_reverse_iterator rend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// [string.view.cstring_view.capacity], capacity
constexpr size_type size() const noexcept;
constexpr size_type length() const noexcept;
constexpr size_type max_size() const noexcept;
[[nodiscard]] constexpr bool empty() const noexcept;

// [string.view.cstring_view.access], element access
constexpr const_reference operator[](size_type pos) const
    pre (pos <= size());
constexpr const_reference at(size_type pos) const;
constexpr const_reference front() const;
constexpr const_reference back() const;
constexpr const_pointer data() const noexcept;
constexpr const_pointer c_str() const noexcept;

// [string.view.cstring_view.modifiers], modifiers
constexpr void remove_prefix(size_type n);
constexpr void remove_suffix(size_type n)
    = delete("cannot remove_suffix in-place on cstring_view while retaining null terminator. Use substr");
constexpr void swap(basic_cstring_view& s) noexcept;

// [string.view.cstring_view.ops], cstring operations
constexpr size_type copy(charT* s, size_type n, size_type pos = 0) const;

constexpr basic_cstring_view<charT, traits> substr(size_type pos = 0) const;
constexpr basic_cstring_view<charT, traits> subview(size_type pos = 0) const;

constexpr basic_string_view<charT, traits> substr(size_type pos, size_type n) const;
constexpr basic_string_view<charT, traits> subview(size_type pos, size_type n) const;

constexpr int compare(basic_string_view<charT, traits> s) const noexcept;
constexpr int compare(size_type pos1, size_type n1, basic_string_view s) const;
constexpr int compare(size_type pos1, size_type n1, basic_string_view s,
    size_type pos2, size_type n2) const;
constexpr int compare(const charT* s) const
    pre (s != nullptr);
constexpr int compare(size_type pos1, size_type n1, const charT* s) const
    pre (s != nullptr);
constexpr int compare(size_type pos1, size_type n1, const charT* s, size_type n2) const
    pre (s != nullptr);

constexpr bool starts_with(basic_string_view<charT, traits> x) const noexcept;
constexpr bool starts_with(charT x) const noexcept;
constexpr bool starts_with(const charT* x) const
    pre (x != nullptr);
constexpr bool ends_with(basic_string_view<charT, traits> x) const noexcept;
constexpr bool ends_with(charT x) const noexcept;
constexpr bool ends_with(const charT* x) const
    pre (x != nullptr);

constexpr bool contains(basic_string_view<charT, traits> x) const noexcept;
constexpr bool contains(charT x) const noexcept;

```

```

constexpr bool contains(const charT* x) const
    pre (x != nullptr);

// [string.view.cstring_view.find], searching
constexpr size_type find(basic_string_view<charT, traits> s, size_type pos = 0) const noexcept;
constexpr size_type find(charT c, size_type pos = 0) const noexcept;
constexpr size_type find(const charT* s, size_type pos, size_type n) const
    pre (s != nullptr);
constexpr size_type find(const charT* s, size_type pos = 0) const
    pre (s != nullptr);
constexpr size_type rfind(basic_string_view<charT, traits> s, size_type pos = npos) const noexcept;
constexpr size_type rfind(charT c, size_type pos = npos) const noexcept;
constexpr size_type rfind(const charT* s, size_type pos, size_type n) const
    pre (s != nullptr);
constexpr size_type rfind(const charT* s, size_type pos = npos) const
    pre (s != nullptr);

constexpr size_type find_first_of(basic_string_view<charT, traits> s, size_type pos = 0) const noexcept;
constexpr size_type find_first_of(charT c, size_type pos = 0) const noexcept;
constexpr size_type find_first_of(const charT* s, size_type pos, size_type n) const
    pre (s != nullptr);
constexpr size_type find_first_of(const charT* s, size_type pos = 0) const
    pre (s != nullptr);
constexpr size_type find_last_of(basic_string_view<charT, traits> s, size_type pos = npos) const noexcept;
constexpr size_type find_last_of(charT c, size_type pos = npos) const noexcept;
constexpr size_type find_last_of(const charT* s, size_type pos, size_type n) const
    pre (s != nullptr);
constexpr size_type find_last_of(const charT* s, size_type pos = npos) const
    pre (s != nullptr);
constexpr size_type find_first_not_of(basic_string_view<charT, traits> s, size_type pos = 0) const noexcept;
constexpr size_type find_first_not_of(charT c, size_type pos = 0) const noexcept;
constexpr size_type find_first_not_of(const charT* s, size_type pos,
                                     size_type n) const
    pre (s != nullptr);
constexpr size_type find_first_not_of(const charT* s, size_type pos = 0) const
    pre (s != nullptr);
constexpr size_type find_last_not_of(basic_string_view<charT, traits> s,
                                     size_type pos = npos) const noexcept;
constexpr size_type find_last_not_of(charT c, size_type pos = npos) const noexcept;
constexpr size_type find_last_not_of(const charT* s, size_type pos,
                                     size_type n) const
    pre (s != nullptr);
constexpr size_type find_last_not_of(const charT* s, size_type pos = npos) const
    pre (s != nullptr);

private:
    const_pointer data_;           // exposition only
    size_type size_;              // exposition only
};
}

```

§ 8.2.2.2 Construction and assignment [string.view.cstring_view.cons]

```
constexpr basic_cstring_view() noexcept;
```

Effects: Constructs an empty `basic_cstring_view`.

Postconditions: `size_` is 0 and `data_` points to a valid empty string

Complexity: $O(1)$.

```
constexpr basic_cstring_view(const charT* str) noexcept;
```

Preconditions: `[str, str + traits::length(str)]` is a valid range, `str[traits::length(str)] == charT()`

Effects: Constructs a `basic_cstring_view` that refers to `str`.

Postconditions: `size_` returns `traits::length(str)` and `data_` returns `str`.

Complexity: $O(\text{traits::length}(\text{str}))$.

```
constexpr basic_cstring_view(const charT* str, size_type len) noexcept;
```

Preconditions: [str, str + len] is a valid range, str[len] == charT().

Effects: Constructs a basic_cstring_view that refers to str.

Postconditions: size_ returns len and data_ returns str.

Complexity: O(1).

```
template<class It, class End>
constexpr basic_cstring_view(It begin, End end);
```

Constraints:

- It satisfies contiguous_iterator.
- End satisfies sized_sentinel_for<It>.
- is_same_v<iter_value_t<It>, charT> is true.
- is_convertible_v<End, size_type> is false.

Preconditions:

- [begin, end] is a valid range.
- It models contiguous_iterator.
- End models sized_sentinel_for<It>.
- *(begin + (end - begin)) == charT()

Effects: Creates a basic_cstring_view that refers to the range from begin to end, inclusive.

Postconditions: size_ returns (end - begin) and data_ returns to_address(begin).

Throws: When and what end - begin throws.

Complexity: O(1).

```
template<class R>
constexpr basic_cstring_view(cstring_like R&& r);
```

Let d be an lvalue of type remove_cvref_t<R>.

Constraints:

- remove_cvref_t<R> is not the same type as basic_cstring_view,
- R models ranges::contiguous_range, ranges::sized_range and cstring_like
- is_same_v<ranges::range_value_t<R>, charT> is true,
- is_convertible_v<R, const charT*> is false, and
- d.operator ::std::basic_cstring_view<charT, traits>() is not a valid expression.

Effects: Creates a basic_cstring_view that refers to the range specified by r.

Postconditions: size_ returns ranges::size(r) and data_ returns ranges::data(r).

Throws: Any exception thrown by ranges::data(r) and ranges::size(r).

§ 8.2.2.3 Deduction guides (string.view.cstring_view.deduct]

```
template<class It, class End>
basic_string_view(It, End) -> basic_string_view<iter_value_t<It>>;
```

Constraints:

- It satisfies contiguous_iterator.
- End satisfies sized_sentinel_for<It>.

```
template<class R>
basic_string_view(cstring_like R&&) -> basic_string_view<ranges::range_value_t<R>>;
```

Constraints: R satisfies ranges::contiguous_range and cstring_like

§ 8.2.2.4 Iterator support [string.view.cstring_view.iterators]

```
constexpr const_iterator begin() const noexcept;  
constexpr const_iterator cbegin() const noexcept;
```

Returns: An iterator such that `addressof(*begin()) == data_`.

```
constexpr const_iterator end() const noexcept;  
constexpr const_iterator cend() const noexcept;
```

Returns: `begin() + size_`.

```
constexpr const_reverse_iterator rbegin() const noexcept;  
constexpr const_reverse_iterator crbegin() const noexcept;
```

Returns: `const_reverse_iterator(end())`.

```
constexpr const_reverse_iterator rend() const noexcept;  
constexpr const_reverse_iterator crend() const noexcept;
```

Returns: `const_reverse_iterator(begin())`.

§ 8.2.2.5 Capacity [string.view.cstring_view.capacity]

```
constexpr size_type size() const noexcept;  
constexpr size_type length() const noexcept;
```

Returns: `size_`.

```
constexpr size_type max_size() const noexcept;
```

Returns: The largest possible number of char-like objects that can be referred to by a `basic_cstring_view`.

```
[[nodiscard]] constexpr bool empty() const noexcept;
```

Returns: `size_ == 0`.

§ 8.2.2.6 Element access [string.view.cstring_view.access]

```
constexpr const_reference operator[](size_type pos) const;
```

Hardened preconditions: `pos <= size_` is true.

[Note: This precondition is identical to `basic_string::operator[]`. — end note]

Returns: `data_[pos]`.

Throws: Nothing.

```
constexpr const_reference at(size_type pos) const;
```

Returns: `data_[pos]`.

Throws: `out_of_range` if `pos > size_`.

```
constexpr const_reference front() const;
```

Hardened preconditions: `empty()` is false.

Returns: `data_[0]`.

Throws: Nothing.

```
constexpr const_reference back() const;
```

Hardened preconditions: `empty()` is false.

Returns: `data_[size_-1]`.

Throws: Nothing.

```
constexpr const_pointer data() const noexcept;  
constexpr const_pointer c_str() const noexcept;
```

Returns: data_.

[Note: The pointer returned is safe to pass to any function expecting a null-terminated string]

Postcondition: The returned value can be dereferenced in [0, size()).

§ 8.2.2.7 Modifiers [string.view.cstring_view.modifiers]

```
constexpr void remove_prefix(size_type n);
```

Hardened preconditions: $n \leq \text{size}()$ is true.

Effects: Equivalent to: $\text{data_} += n$; $\text{size_} -= n$;

```
constexpr void swap(basic_cstring_view& s) noexcept;
```

Effects: Exchanges the values of *this and s.

§ 8.2.2.8 String operations [string.view.cstring_view.ops]

```
constexpr size_type copy(charT* s, size_type n, size_type pos = 0) const;
```

Let rlen be the smaller of n and $\text{size}() - \text{pos}$.

Preconditions: $[s, s + \text{rlen})$ is a valid range.

Effects: Equivalent to $\text{traits::copy}(s, \text{data}() + \text{pos}, \text{rlen})$.

Returns: rlen .

Throws: `out_of_range` if $\text{pos} > \text{size}()$.

Complexity: $O(\text{rlen})$.

```
constexpr basic_cstring_view<charT, traits> substr(size_type pos = 0) const;  
constexpr basic_cstring_view<charT, traits> subview(size_type pos = 0) const;
```

Returns: $\text{basic_cstring_view}(\text{data}() + \text{pos}, \text{size}() - \text{pos})$.

Throws: `out_of_range` if $\text{pos} > \text{size}()$.

```
constexpr basic_string_view<charT, traits> substr(size_type pos, size_type n) const;  
constexpr basic_string_view<charT, traits> subview(size_type pos, size_type n) const;
```

Let rlen be the smaller of n and $\text{size}() - \text{pos}$.

Effects: Determines rlen , the effective length of the string to reference.

Returns: $\text{basic_cstring_view}(\text{data}() + \text{pos}, \text{rlen})$.

Throws: `out_of_range` if $\text{pos} > \text{size}()$.

```
constexpr int compare(basic_string_view str) const noexcept;
```

Returns: Equivalent to: $\text{return substr}(0).compare(\text{str})$;

```
constexpr int compare(size_type pos1, size_type n1, basic_string_view str) const;
```

Effects: Equivalent to: $\text{return substr}(\text{pos1}, \text{n1}).compare(\text{str})$;

```
constexpr int compare(size_type pos1, size_type n1, basic_string_view str, size_type pos2, size_type n2) const;
```

Effects: Equivalent to: $\text{return substr}(\text{pos1}, \text{n1}).compare(\text{str.substr}(\text{pos2}, \text{n2}))$;

```
constexpr int compare(const charT* s) const;
```

Effects: Equivalent to: $\text{return compare}(\text{basic_string_view}<\text{charT}, \text{traits}>(\text{s}))$;

```
constexpr int compare(size_type pos1, size_type n1, const charT* s) const;
```

Effects: Equivalent to: $\text{return substr}(\text{pos1}, \text{n1}).compare(\text{basic_string_view}<\text{charT}, \text{traits}>(\text{s}))$;

```
constexpr int compare(size_type pos1, size_type n1, const charT* s, size_type n2) const;
```

Effects: Equivalent to: $\text{return substr}(\text{pos1}, \text{n1}).compare(\text{basic_string_view}<\text{charT}, \text{traits}>(\text{s}, \text{n2}))$;

```
constexpr bool starts_with(basic_string_view<charT, traits> x) const noexcept;
```

Let `rlen` be the smaller of `size()` and `x.size()`.

Effects: Equivalent to: `return basic_string_view<charT, traits>(data(), rlen) == x;`

```
constexpr bool starts_with(charT x) const noexcept;
```

Effects: Equivalent to: `return !empty() && traits::eq(front(), x);`

```
constexpr bool starts_with(const charT* x) const;
```

Effects: Equivalent to: `return starts_with(basic_string_view<charT, traits>(x));`

```
constexpr bool ends_with(basic_string_view<charT, traits> x) const noexcept;
```

Let `rlen` be the smaller of `size()` and `x.size()`.

Effects: Equivalent to: `return basic_string_view(data() + (size() - rlen), rlen) == x;`

```
constexpr bool ends_with(charT x) const noexcept;
```

Effects: Equivalent to: `return !empty() && traits::eq(back(), x);`

```
constexpr bool ends_with(const charT* x) const
```

Effects: Equivalent to: `return ends_with(basic_string_view(x));`

```
constexpr bool contains(basic_string_view<charT, traits> x) const noexcept;  
constexpr bool contains(charT x) const noexcept;  
constexpr bool contains(const charT* x) const  
    pre (x != nullptr);
```

Effects: Equivalent to: `return find(x) != npos;`

§ 8.2.2.9 Searching [`string.view.cstring_view.find`]

Member functions in this subclause have complexity $O(\text{size()} * \text{str.size}())$ at worst, although implementations should do better.

Let `F` be one of `find`, `rfind`, `find_first_of`, `find_last_of`, `find_first_not_of`, and `find_last_not_of`.

— Each member function of the form

```
constexpr return-type F (const charT* s, size_type pos) const;
```

has effects equivalent to: `return F(basic_string_view(s), pos);`

— Each member function of the form

```
constexpr return-type F (const charT* s, size_type pos, size_type n) const;
```

has effects equivalent to: `return F(basic_string_view(s, n), pos);`

— Each member function of the form

```
constexpr return-type F (charT c, size_type pos) const noexcept;
```

has effects equivalent to: `return F(basic_string_view(addressof(c), 1), pos);`

— Each member function of the form

```
constexpr return-type F (basic_string_view<charT, traits> s, size_type pos = 0) const noexcept;
```

has effects equivalent to: `return basic_string_view<charT, traits>(*this).F(s, pos);`

§ 8.2.3 Non-member comparison functions, [`string.view.cstring_view.comparison`]

```
template<class charT, class traits>  
constexpr bool operator==(basic_cstring_view<charT, traits> x,  
    type_identity_t<basic_cstring_view<charT, traits>> y) noexcept;
```

Returns: `lhs.compare(rhs) == 0`.

```
template<class charT, class traits>  
constexpr /* see below */  
operator<=>(basic_cstring_view<charT, traits> x,  
    type_identity_t<basic_cstring_view<charT, traits>> y) noexcept;
```

Returns: Equivalent to: `return lhs.substr().operator<=>(rhs);`

§ 8.2.4 Inserters and extractors, [string.view.cstring_view.io]

```
template<class charT, class traits>
    basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os,
                    basic_cstring_view<charT, traits> str);
```

Returns: Equivalent to: `return os << str.substr();`

§ 8.2.5 Hash support, [string.view.cstring_view.hash]

```
namespace std {
    template<> struct hash<cstring_view>;
    template<> struct hash<u8cstring_view>;
    template<> struct hash<u16cstring_view>;
    template<> struct hash<u32cstring_view>;
    template<> struct hash<wcstring_view>;
}
```

The specialization is enabled.

[Note: The hash value of a string view object is equal to the hash value of the corresponding string object [basic.string.hash]. — end note]

§ 8.2.6 Suffix for basic_cstring_view literals, [string.view.cstring_view.literals]

```
constexpr cstring_view operator""csv(const char* str, size_t len) noexcept;
```

Returns: `cstring_view{str, len}`.

```
constexpr u8cstring_view operator""csv(const char8_t* str, size_t len) noexcept;
```

Returns: `u8cstring_view{str, len}`.

```
constexpr u16cstring_view operator""csv(const char16_t* str, size_t len) noexcept;
```

Returns: `u16cstring_view{str, len}`.

```
constexpr u32cstring_view operator""csv(const char32_t* str, size_t len) noexcept;
```

Returns: `u32cstring_view{str, len}`.

```
constexpr wcstring_view operator""csv(const wchar_t* str, size_t len) noexcept;
```

Returns: `wcstring_view{str, len}`.